

# Sequential Workflows in LangGraph

Your first LangGraph code: build real sequential workflows using StateGraph, TypedDict, nodes, edges, and the compile-invoke pattern.

## WHAT THIS CHAPTER COVERS

- ▶ The five-step LangGraph build pattern
- ▶ Defining state with TypedDict
- ▶ Adding nodes (Python functions) to a graph
- ▶ Connecting nodes with directed edges
- ▶ Compiling and invoking a workflow
- ▶ BMI Calculator — a non-LLM sequential workflow
- ▶ LLM QA Workflow — integrating LangChain models
- ▶ Prompt Chaining — multiple sequential LLM calls

# Sequential Workflows in LangGraph

**RECAP — CHAPTERS 1–3**

The first three chapters established the theoretical foundation: what distinguishes Agentic AI from Generative AI, why LangGraph was created to handle the limitations of LangChain’s sequential chains, and LangGraph’s core abstractions — **StateGraph**, **nodes**, **edges**, **conditional edges**, **state**, and **checkpoints**. This chapter is the first purely practical session: every concept is implemented in code.

## 1. Environment Setup

All chapter code runs inside Jupyter Notebooks (.ipynb) inside a single project folder. Jupyter is chosen because LangGraph’s built-in graph visualizer — `workflow.get_graph().draw_mermaid_png()` — only works in notebook environments. For production projects, normal Python files are used instead.

### Required Packages

Four packages are needed for this chapter. Install them once inside a virtual environment:

```
# Create and activate virtual environment first
python -m venv myenv
myenv\Scripts\activate           # Windows
source myenv/bin/activate        # macOS / Linux

# Install required packages
pip install langgraph
pip install langchain
pip install langchain-huggingface # or langchain-openai for OpenAI models
pip install python-dotenv
```

| Package               | Purpose                                                      |
|-----------------------|--------------------------------------------------------------|
| langgraph             | Core library for building graph-based workflows              |
| langchain             | LLM components: prompts, chat models, output parsers, chains |
| langchain-huggingface | HuggingFace model integration (used in this chapter)         |
| python-dotenv         | Load API keys from a .env file at runtime                    |

## IMPORTANT

LangGraph and LangChain are complementary, not competing. LangGraph provides the workflow execution engine (graph, nodes, edges, state). LangChain provides the AI components that run *inside* nodes: chat models, prompt templates, output parsers, and chains. Every LangGraph project that uses an LLM also imports LangChain components.

## Verify Installation

```
from langgraph.graph import StateGraph
# If no ImportError is raised, installation is successful
```

### 🔄 Quick Revision — Environment Setup

- `langgraph` provides the graph engine; `langchain` provides LLM components that run inside nodes.
- Use Jupyter Notebooks for this chapter — the built-in graph visualizer only works in notebooks.
- Store API keys in a `.env` file and load with `load_dotenv()`. Never hardcode secrets.
- Install once per virtual environment; all code in the playlist shares the same project folder.

## 2. The Five-Step LangGraph Build Pattern

Every LangGraph sequential workflow, regardless of complexity, is built using the same five-step pattern. Internalising this sequence removes cognitive overhead when writing new workflows.

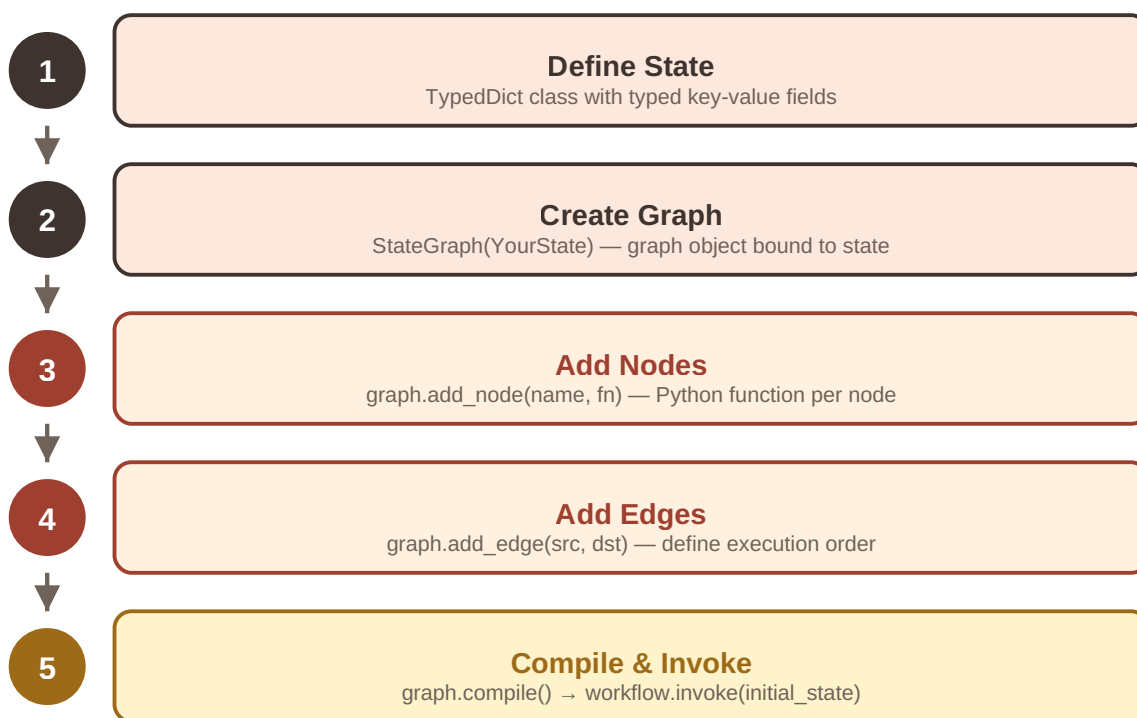


Figure 4.1 — The five universal steps to build any LangGraph sequential workflow.

## Step 1 — Define State

State is a *TypedDict* class — a Python typed dictionary where every key has an explicitly declared type. It is the single shared data object that flows through the entire graph, accumulating updates from every node it passes through.

```
from typing import TypedDict

class MyState(TypedDict):
    field_one: str
    field_two: float
    field_three: int
```

## Step 2 — Create the Graph Object

Instantiate `StateGraph` and bind it to the state class. This registration tells `LangGraph` which state schema every node in this graph must respect.

```
from langgraph.graph import StateGraph

graph = StateGraph(MyState)  # graph is now bound to MyState
```

## Steps 3 & 4 — Add Nodes and Edges

Nodes are Python functions. Register them with `graph.add_node(name, fn)` and connect them with `graph.add_edge(source, destination)`. The `START` and `END` sentinels mark the entry and exit of the graph.

```
from langgraph.graph import StateGraph, START, END

graph.add_node("node_name", my_function)  # register node
graph.add_edge(START, "node_name")        # connect START to node
graph.add_edge("node_name", END)          # connect node to END
```

## Step 5 — Compile and Invoke

`graph.compile()` validates the graph structure and returns a *Runnable* workflow object. Invoke the workflow by passing an initial state dictionary; the output is the final state after all nodes have executed.

```
workflow = graph.compile()

initial_state = {"field_one": "hello", "field_two": 1.0}
final_state = workflow.invoke(initial_state)
# final_state is the complete TypedDict after all node executions
```

## IMPORTANT

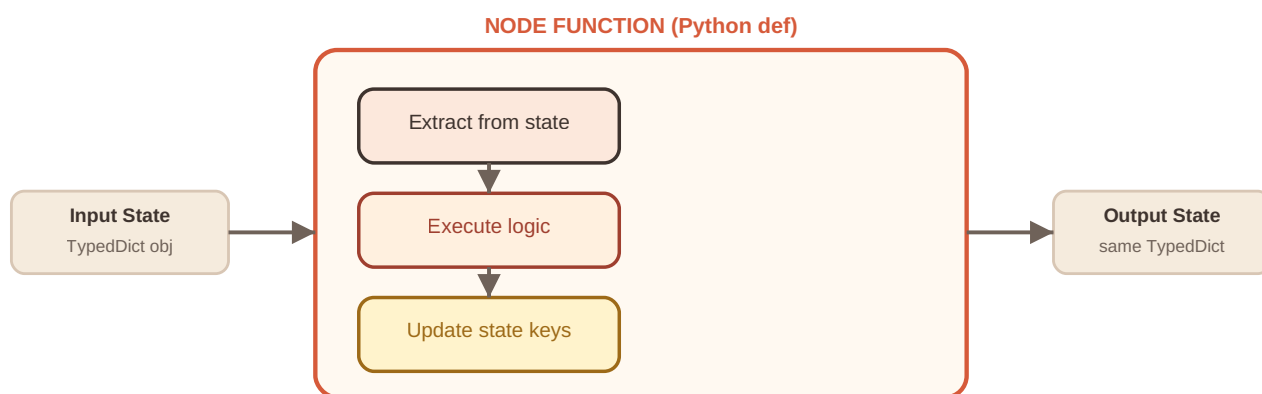
START and END are virtual sentinel nodes — they do not execute any code. START marks where control enters the graph; END marks where it exits and the final state is returned. They must be connected via `add_edge` just like real nodes.

### 🔄 Quick Revision — Five-Step Pattern

- Step 1 — define state as a `TypedDict` class with typed fields.
- Step 2 — create the graph: `StateGraph(YourState)`.
- Step 3 — add nodes: `graph.add_node(name, fn)` mapping string names to Python functions.
- Step 4 — add edges: `graph.add_edge(src, dst)` using START and END sentinels.
- Step 5 — compile with `graph.compile()`, then invoke with `workflow.invoke(initial_state)`.

## 3. Node Anatomy — How a Node Function Works

Every node in a LangGraph graph is a regular Python function with a specific signature: it receives the current state object and returns an updated state object. This contract is the fundamental mechanism by which data moves through a workflow.



*Every node receives the full state and returns the (partially updated) state.*

*Figure 4.2 — Internal anatomy of a LangGraph node function: extract, process, return updated state.*

## Node Function Signature

```
def my_node(state: MyState) -> MyState:
    # 1. Extract needed values from state
    value = state["some_key"]

    # 2. Perform computation or LLM call
    result = compute(value)

    # 3. Write result back into state (partial update)
    state["output_key"] = result

    # 4. Return the updated state
    return state
```

### IMPORTANT

Nodes perform *partial updates* to state — they only modify the keys they care about. Other keys remain unchanged. The returned state is the same TypedDict object with some keys updated. Nodes do not create a new state from scratch.

### INTERVIEW TIP

If asked “what does a LangGraph node receive and return?”: a node receives the *current state* of the graph (a TypedDict instance) and returns the *updated state* (same TypedDict type, with zero or more keys modified). The state is automatically passed to each node by the LangGraph runtime — the developer does not manage this routing manually.

### 🔄 Quick Revision — Node Anatomy

- A node is a Python function: `def fn(state: YourState) -> YourState`.
- Nodes receive the full current state and return the full (partially modified) state.
- Only modify the keys your node is responsible for; leave other keys untouched.
- LangGraph passes the state automatically — you never call node functions directly.
- Type hints are optional but strongly recommended for clarity and IDE support.

## 4. Workflow 1 — BMI Calculator (Non-LLM)

The first workflow is intentionally non-LLM: the goal is to learn LangGraph’s syntax and execution model without the complexity of an AI model. The workflow calculates a person’s BMI from weight and height, then classifies the result into a health category.

## Workflow Design

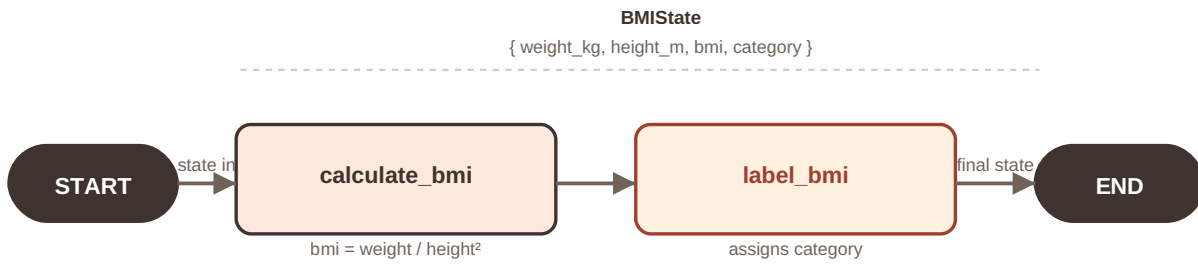


Figure 4.3 — BMI Calculator workflow: two sequential nodes process state between START and END.

## State Definition

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class BMIState(TypedDict):
    weight_kg : float
    height_m : float
    bmi : float
    category : str
```

## Node 1 — calculate\_bmi

```
def calculate_bmi(state: BMIState) -> BMIState:
    bmi = state["weight_kg"] / (state["height_m"] ** 2)
    state["bmi"] = round(bmi, 2)
    return state
```

## HOW IT WORKS

The BMI formula is  $\text{weight (kg)} / \text{height}^2 \text{ (m}^2\text{)}$ . The node reads `weight_kg` and `height_m` from state, computes the result, rounds it to two decimal places, and writes it back into `state["bmi"]`.

## Node 2 — label\_bmi

```
def label_bmi(state: BMIState) -> BMIState:
    bmi = state["bmi"]
    if bmi < 18.5:
        state["category"] = "Underweight"
    elif 18.5 <= bmi < 25:
        state["category"] = "Normal"
    elif 25 <= bmi < 30:
        state["category"] = "Overweight"
    else:
        state["category"] = "Obese"
    return state
```

### HOW IT WORKS

This node reads the already-computed `bmi` key from `state` (populated by the previous node) and uses WHO BMI thresholds to assign a string category to `state["category"]`.

## Graph Assembly and Execution

```
# Build and connect the graph
graph = StateGraph(BMIState)

graph.add_node("calculate_bmi", calculate_bmi)
graph.add_node("label_bmi", label_bmi)

graph.add_edge(START, "calculate_bmi")
graph.add_edge("calculate_bmi", "label_bmi")
graph.add_edge("label_bmi", END)

workflow = graph.compile()

# Execute with initial state — bmi and category are filled by nodes
initial_state = {"weight_kg": 80, "height_m": 1.79}
final_state = workflow.invoke(initial_state)
print(final_state)
# Output: {'weight_kg': 80, 'height_m': 1.79, 'bmi': 24.98, 'category': 'Normal'}
```

## Visualising the Graph

LangGraph provides a built-in Mermaid-based visualizer, available in Jupyter notebooks:

```
from IPython.display import Image
Image(workflow.get_graph().draw_mermaid_png())
```



### BEST PRACTICE

Always name nodes and functions descriptively (e.g. `calculate_bmi`, not `node1`). Node names appear in the graph visualizer and in runtime logs, making debugging significantly easier as workflows grow. The node name and the backing function name can differ, but keeping them the same reduces confusion.

### COMMON MISTAKE

Forgetting to add edges for *every* segment of the path — including the edges from **START** and to **END**. Calling `graph.compile()` will raise a validation error if any node is unreachable or if the graph has no defined entry or exit point.

### 🔄 Quick Revision — BMI Workflow

- Initial state only needs the known values (`weight_kg`, `height_m`); output keys (`bmi`, `category`) are filled in by nodes.
- `calculate_bmi` reads weight and height, writes `bmi`. `label_bmi` reads `bmi`, writes `category`.
- Nodes execute in the order defined by edges — not by the order of `add_node` calls.
- `workflow.invoke(initial_state)` returns the complete final state after all nodes have run.
- Use `workflow.get_graph().draw_mermaid_png()` inside Jupyter to visualise the graph.

## 5. Workflow 2 — LLM Question Answering

The second workflow introduces LangChain model integration. A single node receives a question from state, queries an LLM, and writes the answer back into state. The architecture pattern is identical to the BMI workflow; only the node logic changes.

### Key Detail — PromptTemplate and LCEL Chains

Rather than formatting strings manually, this workflow uses a LangChain *PromptTemplate* and builds a chain using the pipe operator (`|`). This is the standard LangChain Expression Language (LCEL) pattern for composing components.

## Complete Implementation

```
from langgraph.graph import StateGraph, START, END
from langchain_core.prompts import PromptTemplate
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from typing import TypedDict
from dotenv import load_dotenv

load_dotenv()  # reads HUGGINGFACEHUB_API_TOKEN from .env

# Model setup
model = ChatHuggingFace(
    llm=HuggingFaceEndpoint(repo_id="Qwen/Qwen3.8-2.4T-A95B")
)

# Prompt template
prompt = PromptTemplate(
    template="Answer the following question : \n {ques}",
    input_variables=["ques"]
)

# State
class QAState(TypedDict):
    ques : str
    ans : str

# Node function
def get_ans(state: QAState) -> QAState:
    ques = state["ques"]
    chain = prompt | model  # LCEL chain: prompt feeds into model
    ans = chain.invoke({"ques": ques})
    state["ans"] = ans.content  # extract text from AIMessage object
    return state

# Graph assembly
graph = StateGraph(QAState)
graph.add_node("get_ans", get_ans)
graph.add_edge(START, "get_ans")
graph.add_edge("get_ans", END)
workflow = graph.compile()

# Execute
initial_state = {"ques": "What is the distance between earth and moon?"}
final_state = workflow.invoke(initial_state)
print(final_state["ans"])
```

### WHY .CONTENT?

The model returns an `AIMessage` object, not a plain string. The text is in its `.content` attribute. Using a *StrOutputParser* at the end of the chain (`prompt | model | StrOutputParser()`) would extract this automatically — as shown in the prompt chaining workflow in Section 6.

## REAL WORLD INSIGHT

LangGraph adds overhead for a single-node workflow like this one — a direct `chain.invoke(...)` call would be simpler. The value of LangGraph appears when multiple nodes, branching, looping, or checkpointing are needed. Single-node examples exist purely to teach the framework's mechanics in isolation before adding complexity.

### 🔄 Quick Revision — LLM QA Workflow

- LangChain components (PromptTemplate, model) live *inside* a node function — LangGraph orchestrates which node runs when.
- LCEL pipe operator (`|`) composes LangChain components: `prompt | model | parser`.
- LLM responses are `AIMessage` objects; access the text via `.content` or use `StrOutputParser`.
- Store API keys in `.env` and load with `load_dotenv()`; never expose them in source code.
- The workflow returns the full `QASState`; access individual fields with `final_state["ans"]`.

## 6. Workflow 3 — Prompt Chaining (Multiple LLM Calls)

*Prompt chaining* means decomposing a complex task into a sequence of LLM calls, where the output of each call feeds as context into the next. This workflow implements a two-step blog generator: the LLM first produces a detailed outline, then uses that outline to write a full blog post.

### Why Use Prompt Chaining?

A single prompt asking the LLM to write a complete blog post often produces lower-quality output than breaking the task into structured steps. Forcing the model to commit to an outline first creates a scaffold that improves coherence in the final output. This decomposition strategy is a fundamental Agentic AI pattern.

**PROMPT CHAINING — each node calls the LLM once**

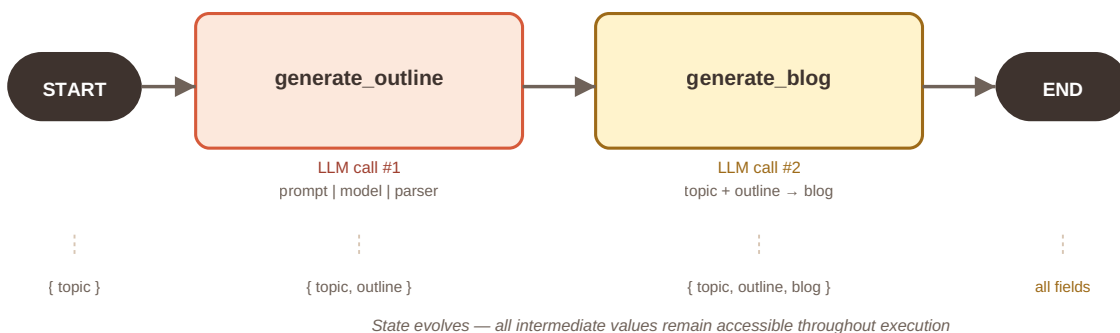


Figure 4.4 — Prompt chaining: two sequential LLM nodes accumulate output into shared state.

## State Design

```
class BlogState(TypedDict):
    topic    : str    # provided by caller — never modified by nodes
    outline  : str    # written by generate_outline node
    blog     : str    # written by generate_blog node
```

## Prompt Templates and Parser

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()  # auto-extracts .content from AIMessage

outline_prompt = PromptTemplate(
    template="Generate a detailed outline on the topic : {topic}",
    input_variables=["topic"]
)

blog_prompt = PromptTemplate(
    template="Generate a blog on topic : {topic} using the given outline : \n {outline}",
    input_variables=["topic", "outline"]
)
```

## Node 1 — generate\_outline

```
def generate_outline(state: BlogState) -> BlogState:
    topic = state["topic"]
    chain = outline_prompt | model | parser  # chain returns plain string
    state["outline"] = chain.invoke({"topic": topic})
    return state
```

## Node 2 — generate\_blog

```
def generate_blog(state: BlogState) -> BlogState:
    outline = state["outline"]  # available because generate_outline ran first
    topic   = state["topic"]
    chain   = blog_prompt | model | parser
    state["blog"] = chain.invoke({"topic": topic, "outline": outline})
    return state
```

## Graph Assembly and Execution

```
graph = StateGraph(BlogState)

graph.add_node("generate_outline", generate_outline)
graph.add_node("generate_blog", generate_blog)

graph.add_edge(START, "generate_outline")
graph.add_edge("generate_outline", "generate_blog")
graph.add_edge("generate_blog", END)

workflow = graph.compile()

initial_state = {"topic": "Rise of AI in India"}
final_state = workflow.invoke(initial_state)

# All intermediate values remain accessible at the end
print(final_state["outline"]) # the generated outline
print(final_state["blog"])   # the full blog post
```

## State Advantage Over LangChain Chains

In LangChain's traditional sequential chains, only the *final* output is returned. With LangGraph's state model, *every* intermediate value is preserved and accessible after execution. Here, **topic**, **outline**, and **blog** are all available in `final_state`, enabling inspection, logging, or downstream use of any intermediate result.

### IMPORTANT

`StrOutputParser` in the chain (`prompt | model | parser`) automatically extracts the string from the `AIMessage` returned by the model. Without the parser, `chain.invoke()` returns an `AIMessage` object and you must call `.content` manually — as in the QA workflow in Section 5.

### BEST PRACTICE

Use a dedicated `PromptTemplate` for each distinct LLM call in a prompt chain. Separate templates make each prompt independently readable, testable, and modifiable. Avoid building multi-step prompts by string concatenation inside node functions.

### INTERVIEW TIP

Prompt chaining is a frequently tested agentic pattern. Key points: (1) it decomposes complex tasks into sequential LLM calls; (2) each call uses the output of the previous as context; (3) in LangGraph this maps directly to a sequence of nodes connected by edges; (4) the state object preserves all intermediate outputs, unlike traditional sequential chains which only return the final result.

### 🔄 Quick Revision — Prompt Chaining

- Prompt chaining = multiple sequential LLM calls, each using the output of the previous call as context.
- Each LLM call maps to one node; the full sequence is connected by edges in the graph.
- `StrOutputParser` automatically extracts the string content from an `AIMessage`.
- `LangGraph` state preserves *all* intermediate values — not just the final output.
- Use separate `PromptTemplate` instances for each distinct LLM call for clarity and maintainability.

## 7. Comparing All Three Workflows

All three workflows share the same five-step structure. The table below compares them across key dimensions to reinforce the pattern.

| Aspect                | BMI Calculator                                                                            | LLM QA                               | Prompt Chaining                                               |
|-----------------------|-------------------------------------------------------------------------------------------|--------------------------------------|---------------------------------------------------------------|
| State class           | <code>BMISState</code>                                                                    | <code>QAState</code>                 | <code>BlogState</code>                                        |
| State fields          | <code>weight_kg</code> , <code>height_m</code> , <code>bmi</code> , <code>category</code> | <code>ques</code> , <code>ans</code> | <code>topic</code> , <code>outline</code> , <code>blog</code> |
| Node count            | 2                                                                                         | 1                                    | 2                                                             |
| LLM involved          | No                                                                                        | Yes (1 call)                         | Yes (2 calls)                                                 |
| LangChain used        | No                                                                                        | Yes                                  | Yes                                                           |
| Output parser         | N/A                                                                                       | Manual <code>.content</code>         | <code>StrOutputParser</code>                                  |
| Primary learning goal | <code>LangGraph</code> syntax                                                             | LLM integration                      | Chaining pattern + state advantage                            |

### REAL WORLD INSIGHT

Sequential workflows are the simplest `LangGraph` pattern — no branching, no looping. In real agentic applications, you will extend this with conditional edges (routing based on state values) and cycles (looping until a condition is met). The sequential pattern taught in this chapter is the foundation that all more complex patterns build on.

### 🔄 Quick Revision — Workflow Comparison

- All three workflows use the same five-step build pattern regardless of whether an LLM is involved.
- The BMI workflow has no LLM — ideal for learning graph mechanics in isolation.
- The QA workflow demonstrates `LangChain` model integration inside a single node.
- Prompt chaining demonstrates that `LangGraph` state preserves all intermediate values, unlike traditional chains.
- Node count scales with task decomposition; each distinct processing step or LLM call becomes its own node.

## K. Key Takeaways

- **Five-step pattern:** Every LangGraph workflow follows the same sequence — define TypedDict state, create StateGraph, add nodes, add edges, compile and invoke. Memorise this pattern.
- **State is the shared contract:** The TypedDict class defines every field that flows through the graph. All nodes read from and write to the same state object.
- **Nodes are Python functions:** A node receives the current state, performs its logic (including LLM calls via LangChain), updates one or more state keys, and returns the state.
- **Edges define execution order:** `add_edge(src, dst)` determines the sequence. `START` and `END` are virtual sentinels marking the entry and exit of the graph.
- **LangGraph + LangChain work together:** LangGraph provides the workflow engine; LangChain provides LLM components (models, prompts, parsers, chains) that run inside nodes.
- **Prompt chaining:** Complex tasks can be decomposed into multiple sequential LLM calls, each as a separate node. State carries all intermediate results to the end of execution.
- **State advantage:** Unlike traditional LangChain sequential chains, LangGraph preserves all intermediate state values — not just the final output.
- **Graph visualisation:** `workflow.get_graph().draw_mermaid_png()` renders the graph in Jupyter, invaluable for debugging and verifying graph structure.

## R. Revision Sheet

### TypedDict State

A Python TypedDict subclass that defines all key-value fields in the graph. Every node receives and returns an instance of this class — the single shared data container for the entire workflow.

### add\_node(name, fn)

Registers a Python function as a named node. The name is used in `add_edge` and appears in the visualizer. The function signature must be `(state) -> state`.

### StateGraph

`StateGraph(YourState)` creates the graph object bound to a specific state class. All nodes and edges are registered on this object before compilation.

### add\_edge(src, dst)

Connects two nodes with a directed edge, establishing execution order. `START` and `END` are sentinels marking graph entry and exit. Every path must connect from `START` and reach `END`.

### **graph.compile()**

Validates graph structure (all nodes reachable, path from START to END exists) and returns a Runnable workflow object. Raises an error if the graph is malformed.

### **Partial State Update**

Nodes modify only the state keys they are responsible for. Other keys pass through unchanged, allowing state to accumulate data incrementally through a multi-node graph.

### **workflow.invoke(state)**

Executes the compiled graph. Accepts the initial state dictionary; returns the final state dictionary after all nodes have executed.

### **Prompt Chaining Pattern**

A complex task is decomposed into multiple sequential LLM calls, each as a separate node. Each call uses the outputs of prior nodes as context, improving result quality.

## **I. Interview Questions**

- Q1.** What is `TypedDict` and why is it used to define state in LangGraph rather than a regular dictionary?
- Q2.** What is the role of `StateGraph` and why must the state class be passed when instantiating it?
- Q3.** Describe the required signature of a LangGraph node function. What does it receive and what must it return?
- Q4.** What are `START` and `END` in LangGraph, and why must edges connect them to other nodes?
- Q5.** What does `graph.compile()` do, and what kinds of structural errors will it catch?
- Q6.** How does LangChain integrate with LangGraph? Where in the architecture do LangChain components run?
- Q7.** What is prompt chaining, and how does the LangGraph state model enable it more effectively than LangChain's traditional sequential chains?
- Q8.** If you need to call an LLM twice (first for an outline, then for a blog), how many nodes would you define and why?
- Q9.** What is the difference between using `StrOutputParser` in a chain versus accessing `.content` on the model output directly?
- Q10.** Why is LangGraph considered overkill for a simple, single-node, single-LLM-call task? When does it become the right tool?



## F. Further Reading

- **LangGraph Documentation** ([langgraph.dev](https://langgraph.dev)) — Official guide to StateGraph, nodes, edges, state management, and compilation.
- **LangChain Expression Language (LCEL)** ([python.langchain.com/docs/concepts/lcel](https://python.langchain.com/docs/concepts/lcel)) — The pipe-operator composition pattern used inside nodes.
- **TypedDict (PEP 589)** ([docs.python.org/3/library/typing.html](https://docs.python.org/3/library/typing.html)) — How Python enforces typed dictionaries at definition time.
- **HuggingFace Inference Endpoints** ([huggingface.co/docs/inference-endpoints](https://huggingface.co/docs/inference-endpoints)) — Serverless API access to open-source models like Qwen used in this chapter.
- **Prompt Chaining — Anthropic Prompting Guide** ([docs.anthropic.com/en/docs/build-with-claude/prompt-engineering](https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering)) — Decomposition and chaining strategies that map directly to multi-node LangGraph workflows.